

---

# C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

---

## Capítulo 1

Introdução aos Computadores, à Internet e à Web

### **Exercícios (1.3 – 1.9)**

Resolvidos passo a passo, com explicações conceituais,  
no estilo incremental Deitel

## Contents

---

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Exercício 1.3 – Hardware ou Software?</b>                            | <b>2</b>  |
| <b>2</b> | <b>Exercício 1.4 – Linguagem Independente vs. Dependente de Máquina</b> | <b>5</b>  |
| <b>3</b> | <b>Exercício 1.5 – Preenchimento de Lacunas: Unidades do Computador</b> | <b>7</b>  |
| <b>4</b> | <b>Exercício 1.6 – Verdadeiro ou Falso</b>                              | <b>9</b>  |
| <b>5</b> | <b>Exercício 1.7 – stdin, stdout e stderr</b>                           | <b>11</b> |
| <b>6</b> | <b>Exercício 1.8 – Programação Orientada a Objetos</b>                  | <b>13</b> |
| <b>7</b> | <b>Exercício 1.9 – Identificando Linguagens de Programação</b>          | <b>15</b> |

## 1. Exercício 1.3 – Hardware ou Software?

### Enunciado

Categorize os seguintes itens como **hardware** ou **software**:

- a) CPU
- b) compilador C++
- c) ALU
- d) pré-processador C++
- e) unidade de entrada
- f) um programa editor

### Resposta detalhada

#### Fundamentação conceitual:

Antes de categorizar cada item, recordemos as definições fundamentais da Seção 1.2:

- **Hardware:** Os diversos *dispositivos físicos* que compõem um sistema de computador – tudo que pode ser tocado: circuitos, chips, teclados, discos, etc.
- **Software:** Os *programas* executados em um computador – as instruções que controlam o hardware e o fazem realizar tarefas úteis.

| Item                                 | Elemento            | Categoria | Justificativa                                                                                                                                                                                                       |
|--------------------------------------|---------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| a                                    | CPU                 | Hardware  | A <i>Unidade Central de Processamento</i> é um componente físico (chip eletrônico) que coordena todas as operações do computador. É a “seção administrativa” do hardware.                                           |
| b                                    | Compilador C++      | Software  | Um compilador é um <i>programa</i> (conjunto de instruções) que traduz código em linguagem de alto nível (C++) para linguagem de máquina. Reside em disco e é executado na CPU – portanto, é software.              |
| c                                    | ALU                 | Hardware  | A <i>Unidade Lógica e Aritmética</i> é a “seção de processamento” física do computador, responsável por realizar cálculos e comparações. Nos computadores modernos, está integrada fisicamente à CPU no mesmo chip. |
| d                                    | Pré-processador C++ | Software  | O pré-processador é um <i>programa</i> que processa diretivas como <code>#include</code> e <code>#define</code> antes da compilação. É software, pois consiste em instruções executadas pelo computador.            |
| e                                    | Unidade de entrada  | Hardware  | A unidade de entrada é a “seção receptora” física do computador – abrange dispositivos como teclado, mouse, scanner. São componentes físicos, portanto, hardware.                                                   |
| f                                    | Programa editor     | Software  | Um programa editor (como <code>vi</code> , <code>emacs</code> ou o editor do Visual Studio) é um <i>programa</i> – uma coleção de instruções de software – usado para criar e modificar o código-fonte.             |
| <b>Resumo:</b> Hardware → a), c), e) |                     |           | Software → b), d), f)                                                                                                                                                                                               |

**Boa prática de programação**

A distinção hardware/software é fundamental em programação. Todo programa que você escreverá em C é *software*, mas seu funcionamento depende inteiramente do *hardware* subjacente. Entender essa relação o tornará um programador mais consciente sobre desempenho, portabilidade e limitações do sistema.

## 2. Exercício 1.4 – Linguagem Independente vs. Dependente de Máquina

### Enunciado

Por que você escreveria um programa em uma linguagem independente de máquina em vez de em uma linguagem dependente da máquina? Por que uma linguagem dependente da máquina poderia ser mais apropriada para a escrita de certos tipos de programas?

### Resposta detalhada

#### Parte 1: Por que usar linguagem independente de máquina?

As **linguagens de alto nível**, como C, são consideradas *independentes de máquina* porque um mesmo programa pode ser compilado e executado em diferentes tipos de computador (com diferentes CPUs e sistemas operacionais), geralmente com poucas ou nenhuma modificação.

As principais razões para escolher uma linguagem independente de máquina são:

- a) **Portabilidade:** Um programa escrito em C pode ser compilado para rodar em um PC com Windows, em um servidor Linux, em um Mac ou em um microcontrolador – o mesmo código-fonte funciona em múltiplas plataformas. Como afirma a Dica de Portabilidade 1.1 do livro: *“os aplicativos escritos em C podem ser executados com pouca ou nenhuma modificação em uma grande variedade de sistemas de computação diferentes”*.
- b) **Facilidade de escrita e leitura:** As linguagens de alto nível usam palavras próximas ao inglês e notação matemática. Um comando como `valorTotal = salario + horaExtra;` é imediatamente compreensível para um humano, ao contrário de uma sequência binária como `+1300042774`.
- c) **Produtividade:** Uma única instrução de alto nível pode equivaler a dezenas de instruções em linguagem de máquina ou simbólica. Isso reduz drasticamente o tempo de desenvolvimento de programas.
- d) **Manutenibilidade:** Programas em linguagens de alto nível são muito mais fáceis de ler, depurar, modificar e manter ao longo do tempo.
- e) **Reutilização:** É possível aproveitar a biblioteca-padrão de C e outras bibliotecas já escritas, sem se preocupar com os detalhes do hardware específico.

#### Parte 2: Quando uma linguagem dependente de máquina pode ser mais apropriada?

As **linguagens dependentes de máquina**, como a linguagem de máquina e a linguagem simbólica (assembly), apesar de difíceis de usar, oferecem vantagens em situações específicas:

- a) **Desempenho crítico:** O programador tem controle absoluto sobre cada instrução executada pelo processador. Em aplicações onde cada

nanossegundo importa (como drivers de dispositivos, kernels de sistemas operacionais e código de tempo real), o assembly pode gerar código mais rápido que qualquer compilador.

- b) **Acesso direto ao hardware:** Em programação de sistemas embarcados (microcontroladores, dispositivos de IoT), às vezes é necessário manipular registradores, portas de I/O e interrupções diretamente – operações que somente linguagens dependentes de máquina permitem com precisão absoluta.
- c) **Tamanho mínimo de código:** Em sistemas com memória extremamente limitada, o código assembly tende a ser menor que o equivalente compilado.
- d) **Inicialização de hardware:** O código de inicialização (“bootloader”) de um computador muitas vezes precisa ser escrito em assembly, antes que qualquer ambiente de linguagem de alto nível esteja disponível.

**Conclusão:** Para a grande maioria das aplicações, a linguagem de alto nível (como C) é a escolha ideal, pois combina portabilidade, produtividade e legibilidade. O assembly fica reservado para as raras situações em que o controle direto do hardware ou o desempenho máximo são requisitos inegociáveis.

#### Dica de portabilidade

Curiosamente, C ocupa uma posição única: é uma linguagem de alto nível com capacidades de baixo nível. Por meio de ponteiros e outros recursos (estudados no Capítulo 7), C permite acesso próximo ao hardware sem abrir mão da portabilidade – razão pela qual ainda é a linguagem predominante em sistemas operacionais e firmware.

### 3. Exercício 1.5 – Preenchimento de Lacunas: Unidades do Computador

#### Enunciado

Preencha os espaços em cada uma das seguintes sentenças:

- a) Qual unidade lógica do computador recebe informações de fora para serem usadas por ele? \_\_\_\_\_.
- b) O processo de instruir o computador para resolver problemas específicos é chamado de \_\_\_\_\_.
- c) Que tipo de linguagem de computador usa abreviações em inglês para as instruções em linguagem de máquina? \_\_\_\_\_.
- d) Qual unidade lógica do computador envia informações que já foram processadas por ele para diversos dispositivos? \_\_\_\_\_.
- e) Quais unidades lógicas do computador retêm informações? \_\_\_\_\_.
- f) Qual unidade lógica do computador realiza cálculos? \_\_\_\_\_.
- g) Qual unidade lógica do computador toma decisões lógicas? \_\_\_\_\_.
- h) O nível de linguagem de computador mais conveniente para a elaboração rápida e fácil de programas é \_\_\_\_\_.
- i) A única linguagem que um computador entende com clareza é a chamada \_\_\_\_\_ desse computador.
- j) Qual unidade lógica do computador coordena as atividades de todas as outras unidades lógicas? \_\_\_\_\_.

#### Resposta detalhada

**a) Unidade de entrada**

A *unidade de entrada* é a seção “receptora” do computador (Seção 1.3). Ela capta informações do mundo exterior – teclado, mouse, microfone, scanner, sensores, dados da Internet – e as disponibiliza para as demais unidades processarem. Sem ela, o computador não teria como receber dados ou instruções.

**b) Programação**

Programação é o processo pelo qual os programadores criam conjuntos ordenados de instruções (programas) que instruem o computador a resolver problemas específicos. Todo o conteúdo deste livro é dedicado a ensiná-lo a programar em C.

**c) Linguagem simbólica** (também chamada de *assembly*)

Da Seção 1.6: as linguagens simbólicas substituem as sequências numéricas da linguagem de máquina por abreviações mnemônicas próximas ao inglês, como `load`, `add`, `store`. Programas chamados *assemblers* (montadores) traduzem esse código para linguagem de máquina.

**d) Unidade de saída**

A *unidade de saída* é a seção de “expedição” do computador. Ela pega as informações já processadas e as envia para dispositivos externos – monitor, impressora, alto-falantes, arquivos em disco, conexões de rede, etc.

**e) Unidade de memória e unidade de armazenamento secundário**

Duas unidades retêm informações:

- A **unidade de memória** (RAM) armazena dados temporariamente enquanto o programa é executado. É volátil – perde seu conteúdo quando o computador é desligado.
- A **unidade de armazenamento secundário** (disco rígido, SSD, etc.) armazena dados de forma persistente e por longos períodos.

**f) Unidade lógica e aritmética (ALU)**

A ALU é a seção de “processamento” do computador. Ela executa as quatro operações aritméticas básicas (adição, subtração, multiplicação e divisão), bem como operações lógicas (comparações entre valores). Nos processadores modernos, a ALU está integrada dentro da CPU.

**g) Unidade lógica e aritmética (ALU)**

A mesma ALU que realiza cálculos também é responsável por *tomar decisões lógicas* – como comparar se dois valores são iguais, se um é maior que outro, etc. Essas decisões são a base de todas as estruturas de controle que você aprenderá em C (`if`, `while`, `for`, etc.).

**h) Linguagem de alto nível**

As linguagens de alto nível (como C, C++, Java) são as mais convenientes para o programador, pois usam sintaxe próxima ao inglês e permitem expressar operações complexas em poucas instruções. Uma única instrução de alto nível pode corresponder a dezenas de instruções de máquina.

**i) Linguagem de máquina**

A linguagem de máquina é o “idioma nativo” de cada processador específico. É a única linguagem que a CPU entende diretamente, sem necessidade de tradução. Consiste em sequências de números binários que representam as operações elementares do hardware.

**j) Unidade central de processamento (CPU)**

A CPU é a seção “administrativa” do computador. Ela coordena e supervisiona o funcionamento de todas as demais unidades lógicas: diz à unidade de entrada quando ler dados, à ALU quando calcular, à unidade de saída quando enviar resultados, e ao armazenamento secundário quando gravar ou recuperar dados.

#### 4. Exercício 1.6 – Verdadeiro ou Falso

##### Enunciado

Indique se as sentenças a seguir são **verdadeiras** ou **falsas**. Explique sua resposta caso a resposta seja *falsa*.

- a) Linguagens de máquina geralmente são dependentes da máquina.
- b) Assim como outras linguagens de alto nível, C geralmente é considerada como independente da máquina.

##### Item (a) – Linguagens de máquina são dependentes da máquina

**Sentença: VERDADEIRA**

**Explicação:**

Da Seção 1.6: “*As linguagens de máquina são dependentes da máquina, isto é, uma linguagem de máquina em particular só pode ser usada em um tipo de computador.*”

Cada família de processadores (Intel x86-64, ARM, RISC-V, MIPS, etc.) possui seu próprio conjunto de instruções de máquina. Um programa em linguagem de máquina escrito para um processador Intel não funcionará em um processador ARM, e vice-versa. Isso ocorre porque a linguagem de máquina é definida pelo projeto físico do hardware do processador – pelos circuitos internos que interpretam cada sequência de bits como uma operação específica.

Essa dependência de máquina é exatamente a razão histórica pela qual as linguagens de alto nível foram criadas: para libertar os programadores da necessidade de reescrever seus programas inteiramente a cada mudança de hardware.

##### Item (b) – C é independente de máquina

**Sentença: VERDADEIRA** (com ressalva importante)

**Explicação:**

Da Seção 1.7 e da Dica de Portabilidade 1.1: C é *predominantemente* independente de hardware. Um programa bem escrito em C pode ser compilado e executado em múltiplas plataformas (Windows, Linux, macOS, sistemas embarcados) com poucas ou nenhuma modificação.

**A ressalva essencial – Dica de Portabilidade 1.2:**

**Dica de portabilidade**

Embora seja possível escrever programas portáveis em C, existem muitos problemas entre compiladores C diferentes e computadores diferentes que podem tornar a portabilidade difícil de ser alcançada. Simplesmente escrever programas em C *não garante* portabilidade. Com frequência, o programador precisará lidar com variações entre computadores.

Exemplos de situações que podem comprometer a portabilidade em C:

- O tamanho em bytes de tipos como `int` e `long` varia entre plataformas (32 ou 64 bits).
- Extensões proprietárias de compiladores (recursos específicos do MSVC ou do GCC que não estão no padrão C).
- Dependência de comportamento indefinido (*undefined behavior*) do padrão C.

## 5. Exercício 1.7 – stdin, stdout e stderr

### Enunciado

Explique o significado dos termos a seguir:

- a) `stdin`
- b) `stdout`
- c) `stderr`

### Resposta detalhada

Os três termos descrevem os três **fluxos de dados padrão** (*standard streams*) disponíveis em todo programa C que é executado em um sistema operacional baseado em UNIX/Linux (e também em Windows). Eles são introduzidos na Seção 1.14. Cada fluxo é um canal de comunicação predefinido entre o programa e o sistema operacional.

#### a) `stdin` – Fluxo de Entrada Padrão (*Standard Input*)

- **O que é:** O canal pelo qual o programa recebe dados de entrada.
- **Dispositivo padrão:** O teclado. Quando seu programa em C chama `scanf()` ou `getchar()`, ele lê dados do `stdin`.
- **Redirecionamento:** O `stdin` pode ser redirecionado para ler de um arquivo em vez do teclado. Por exemplo, no terminal Linux:

```
./meu_programa < dados.txt
```

Isso faz o programa ler de `dados.txt` como se o usuário tivesse digitado os dados pelo teclado.

- **Origem do nome:** “`stdin`” = *standard input* (entrada padrão).

#### b) `stdout` – Fluxo de Saída Padrão (*Standard Output*)

- **O que é:** O canal pelo qual o programa envia sua saída normal (resultados, mensagens, dados calculados).
- **Dispositivo padrão:** A tela (monitor). Quando seu programa em C chama `printf()`, ele escreve no `stdout`.
- **Redirecionamento:** O `stdout` pode ser redirecionado para gravar em um arquivo:

```
./meu_programa > resultado.txt
```

Isso salva toda a saída do programa no arquivo `resultado.txt`.

- **Combinação:** É possível redirecionar tanto entrada quanto saída simultaneamente:

```
./meu_programa < dados.txt > resultado.txt
```

#### c) `stderr` – Fluxo de Erro Padrão (*Standard Error*)

- **O que é:** Um canal separado do `stdout`, destinado exclusivamente a *mensagens de erro e diagnóstico*.
- **Dispositivo padrão:** Também a tela, por padrão.

- **Por que um canal separado?** A grande vantagem de ter `stderr` independente do `stdout` é que o usuário pode redirecionar a saída normal (`stdout`) para um arquivo, enquanto as mensagens de erro (`stderr`) continuam aparecendo na tela – garantindo que erros nunca passem despercebidos:

```
./meu_programa > resultado.txt    /* stdout vai para
                                   arquivo */
                                   /* stderr continua
                                   na tela */
```

- **Uso em C:** A função `fprintf(stderr, "Mensagem de erro");` é usada para escrever no `stderr`. Isso será estudado em detalhes nos capítulos sobre funções e entrada/saída formatada.

#### Resumo visual:

| Stream              | Tipo           | Padrão  | Função C                                       |
|---------------------|----------------|---------|------------------------------------------------|
| <code>stdin</code>  | Entrada        | Teclado | <code>scanf()</code> , <code>getchar()</code>  |
| <code>stdout</code> | Saída normal   | Tela    | <code>printf()</code> , <code>putchar()</code> |
| <code>stderr</code> | Saída de erros | Tela    | <code>fprintf(stderr, ...)</code>              |

## 6. Exercício 1.8 – Programação Orientada a Objetos

### Enunciado

Por que tanta atenção é dada hoje à programação orientada a objetos?

### Resposta detalhada

Esta é uma questão que vai além de uma resposta técnica simples – ela toca na história e na evolução da engenharia de software. Vamos construir a resposta incrementalmente, a partir do que o Capítulo 1 nos ensina nas Seções 1.9 e 1.13.

#### 1. O problema que a POO veio resolver

Na Seção 1.13, o co-autor Harvey Deitel relata que, já na década de 1960, as organizações de software enfrentavam graves dificuldades com projetos de grande porte: softwares com atrasos de entrega, estouro de orçamento e resultados pouco confiáveis. As técnicas de *programação procedural* (como C, FORTRAN, COBOL) focavam em **ações (verbos)**, o que tornava difícil modelar o mundo real – que é composto de **coisas (substantivos)**.

#### 2. O que a Programação Orientada a Objetos (POO) oferece

A POO organiza o software em torno de **objetos** – componentes que modelam entidades do mundo real (uma conta bancária, um produto, um aluno) e encapsulam tanto seus *dados* quanto seus *comportamentos*. As principais razões pelas quais a POO recebe tanta atenção são:

- a) **Reutilização de software:** Objetos bem projetados (chamados *classes*) tendem a ser reutilizáveis em projetos futuros. Isso economiza tempo e dinheiro, evitando que as equipes “reinventem a roda” a cada novo projeto. A Observação sobre Engenharia de Software 1.1 do livro destaca que grandes bibliotecas de classes reutilizáveis estão disponíveis gratuitamente na Internet.
- b) **Software mais inteligível e organizado:** Como afirma a Seção 1.13, algumas organizações relatam que o principal benefício da POO *não* é a reutilização, mas sim que o software produzido é mais *claro, bem organizado e fácil de manter, modificar e depurar*. Isso é vital porque até 80% dos custos de software estão associados à *manutenção contínua*, não ao desenvolvimento inicial.
- c) **Modelagem natural do mundo real:** Vivemos em um mundo de objetos (carros, pessoas, produtos, contratos). A POO permite que os programadores modelem o sistema de forma que espelhe essa realidade, tornando o design mais intuitivo.
- d) **Produtividade das equipes:** O uso de uma abordagem modular e orientada a objetos torna os grupos de desenvolvimento muito mais produtivos do que seria possível com técnicas procedurais.
- e) **Encapsulamento e proteção de dados:** A POO permite esconder os detalhes internos de implementação de um objeto, expondo apenas o que é necessário para o uso externo. Isso reduz erros e facilita a evolução do

sistema.

### 3. Perspectiva histórica e o papel de C

C++ (Seção 1.9), desenvolvida por Bjarne Stroustrup no Bell Laboratories, foi a primeira linguagem a trazer POO com ampla aceitação no mercado, ao adicionar recursos de objetos à linguagem C. Desde a década de 1990, linguagens orientadas a objetos (C++, Java, C#, Python) tornaram-se dominantes – e a POO passou a ser a metodologia-chave do desenvolvimento de software.

**Nota importante para este curso:** Você aprenderá primeiro C (Capítulos 1–14), desenvolvendo uma base sólida em programação estruturada. Nos Capítulos 15–24, você então estudará C++ e POO – construindo sobre o alicerce que está construindo agora.

## 7. Exercício 1.9 – Identificando Linguagens de Programação

### Enunciado

Qual linguagem de programação é mais bem descrita por cada uma das seguintes sentenças?

- a) Desenvolvida pela IBM para aplicações científicas e de engenharia.
- b) Desenvolvida especificamente para aplicações comerciais.
- c) Desenvolvida para o ensino de programação estruturada.
- d) Seu nome homenageia a primeira pessoa a programar um computador no mundo.
- e) Desenvolvida para familiarizar iniciantes com as técnicas de programação.
- f) Desenvolvida especificamente para ajudar os programadores a migrar para .NET.
- g) Conhecida como a linguagem de desenvolvimento do UNIX.
- h) Formada, principalmente, pela inclusão da programação orientada a objetos à linguagem C.
- i) Inicialmente, teve sucesso devido à sua capacidade de criar páginas Web com conteúdo dinâmico.

## Resposta detalhada

| Item | Descrição                                                                     | Linguagem        |
|------|-------------------------------------------------------------------------------|------------------|
| a    | Desenvolvida pela IBM para aplicações científicas e de engenharia.            | <b>FORTTRAN</b>  |
| b    | Desenvolvida especificamente para aplicações comerciais.                      | <b>COBOL</b>     |
| c    | Desenvolvida para o ensino de programação estruturada.                        | <b>Pascal</b>    |
| d    | Seu nome homenageia a primeira pessoa a programar um computador no mundo.     | <b>Ada</b>       |
| e    | Desenvolvida para familiarizar iniciantes com as técnicas de programação.     | <b>BASIC</b>     |
| f    | Desenvolvida especificamente para ajudar os programadores a migrar para .NET. | <b>Visual C#</b> |
| g    | Conhecida como a linguagem de desenvolvimento do UNIX.                        | <b>C</b>         |
| h    | Formada, principalmente, pela inclusão da POO à linguagem C.                  | <b>C++</b>       |
| i    | Teve sucesso pela capacidade de criar páginas Web com conteúdo dinâmico.      | <b>Java</b>      |

**Explicações detalhadas:**

**a) FORTRAN (FORmula TRANslator):** Desenvolvida pela IBM Corporation em meados da década de 1950 (Seção 1.11). Foi criada para aplicações científicas e de engenharia que exigem cálculos matemáticos complexos. Ainda é amplamente utilizada em aplicações de engenharia e simulações científicas.

**b) COBOL (COmmon Business Oriented Language):** Desenvolvida no final da década de 1950 por fabricantes de computadores, pelo governo dos EUA e por usuários corporativos (Seção 1.11). É utilizada principalmente em aplicações comerciais que exigem manipulação precisa de grandes volumes de dados (como sistemas bancários e de folha de pagamento). Muitos sistemas legados ainda rodam COBOL hoje.

**c) Pascal:** Projetada pelo professor Niklaus Wirth em 1971, em homenagem ao matemático Blaise Pascal (Seção 1.11). Foi criada especificamente para o ensino de programação estruturada e tornou-se a linguagem preferida em ambientes acadêmicos por décadas.

**d) Ada:** Desenvolvida com o patrocínio do Departamento de Defesa dos EUA na década de 1970 (Seção 1.11). Recebeu o nome de **Lady Ada**

**Lovelace**, filha do poeta Lord Byron, reconhecida historicamente como a primeira programadora do mundo (século XIX). Ada suporta *multitarefa* nativamente.

**e) BASIC (Beginner's All-purpose Symbolic Instruction Code):**

Desenvolvida em meados da década de 1960 no Dartmouth College (Seção 1.12), com o objetivo de familiarizar iniciantes com as técnicas de programação. Foi a linguagem de entrada de inúmeros programadores nas décadas de 1970 e 1980.

**f) Visual C#:** Uma nova linguagem criada pela Microsoft exclusivamente para a plataforma .NET (Seção 1.12), baseada em C++ e Java. Foi desenvolvida especificamente para ajudar os programadores a criar aplicações para o ambiente .NET, facilitando a migração de desenvolvedores C++ e Java para a plataforma Microsoft.

**g) C:** Como já estudamos extensamente no Capítulo 1 (Seção 1.7), C ficou mundialmente conhecida como a linguagem de desenvolvimento do sistema operacional UNIX. Criada por Dennis Ritchie no Bell Laboratories em 1972, C e UNIX cresceram juntos e definiram uma era na computação.

**h) C++:** Desenvolvida por Bjarne Stroustrup no Bell Laboratories (Seção 1.9). C++ foi formada adicionando-se recursos de programação orientada a objetos à linguagem C – especificamente, absorvendo os conceitos de objetos da linguagem de simulação Simula 67.

**i) Java:** Criada pela Sun Microsystems com base em C++ (Seção 1.10). Embora inicialmente desenvolvida para aparelhos eletrônicos, Java ganhou enorme popularidade quando a World Wide Web explodiu em 1993, pois permitia criar *applets* com conteúdo dinâmico e interativo nas páginas Web – uma capacidade revolucionária na época.

## Resumo das Respostas – Exercícios 1.3 a 1.9

### Gabarito Rápido

#### 1.3 – Hardware ou Software:

- a) CPU → **hardware**    b) compilador C++ → **software**    c) ALU → **hardware**  
d) pré-processador C++ → **software**    e) unidade de entrada → **hardware**  
f) programa editor → **software**

#### 1.5 – Unidades do Computador:

- a) unidade de entrada    b) programação    c) linguagem simbólica  
d) unidade de saída    e) memória + armazenamento secundário  
f) ALU    g) ALU    h) linguagem de alto nível  
i) linguagem de máquina    j) CPU

#### 1.6 – Verdadeiro/Falso:

- a) **Verdadeiro** – linguagens de máquina são dependentes da máquina.  
b) **Verdadeiro** (com ressalva) – C é predominantemente independente, mas portabilidade perfeita exige cuidado.

#### 1.7 – Streams:

- `stdin` = fluxo de entrada padrão (teclado)
- `stdout` = fluxo de saída padrão (tela)
- `stderr` = fluxo de erro padrão (tela, separado do `stdout`)

#### 1.9 – Linguagens:

- a) FORTRAN    b) COBOL    c) Pascal    d) Ada    e) BASIC  
f) Visual C#    g) C    h) C++    i) Java

### Boa prática de programação

Você completou os Exercícios do Capítulo 1! Os conceitos aqui revisados – organização do computador, tipos de linguagens, ambiente de desenvolvimento, streams padrão – formam a base conceitual sobre a qual todo o restante do livro é construído. No Capítulo 2, você começará a escrever seus primeiros programas em C, aplicando esses conceitos de forma prática e concreta.